STA 5666
R Solution Code
Mostafa Sameer

Problem 1: Capability Ratios

```
n1 <- 5
xbar1 <- 0.00109
Rbar <- 0.00635
USL1 <- 0.01
LSL1 <- -0.01
target1 <- 0
# Estimate sigma
d2 <- 2.326  # for n=5
sigma_hat <- Rbar / d2
# Capability indices
Cp <- (USL1 - LSL1) / (6 * sigma_hat)
Cpu <- (USL1 - xbar1) / (3 * sigma_hat)
Cpl <- (xbar1 - LSL1) / (3 * sigma_hat)
Cpk <- min(Cpu, Cpl)
V <- (target1 - xbar1) / sigma_hat
Cpm <- Cp / sqrt(1 + V^2)
xi <- (xbar1 - target1) / sigma_hat
Cpkm <- Cpk / sqrt(1 + xi^2)
# Interpretation
cat("Problem 1 Results:\n")
cat(sprintf("Cp = %.3f (Uses %.0f%% of spec band)\n", Cp, 100/Cp))
cat(sprintf("Cpk = %.3f (Process is %s)\n", Cpk, ifelse(Cpk > 1, "capable", "not capable")))
cat(sprintf("Cpm = %.3f (Mean is %s)\n", Cpm, ifelse(Cpm > 1, "in middle third", "off-center")))
cat(sprintf("Cpkm = %.3f (Process is marginally capable)\n\n", Cpkm))
```

Problem 2: Capability Analysis

```
USL2 <- 100
LSL2 <- 90
n2 <- 30
xbar2 <- 97
s2 <- 1.6
# Part (a): Cp point estimate and 95% CI
Cp_hat <- (USL2 - LSL2) / (6 * s2)
chi_low <- qchisq(0.025, n2-1)
chi_high <- qchisq(0.975, n2-1)
CI_Cp_low <- Cp_hat * sqrt(chi_low/(n2-1))
CI_Cp_high <- Cp_hat * sqrt(chi_high/(n2-1))
# Part (c): Test H0: Cp <= 1.33 vs Ha: Cp > 1.33
Cp0 <- 1.33
test_stat <- (n2-1) * (Cp0/Cp_hat)^2
critical_val <- qchisq(0.95, n2-1)
reject_H0 <- test_stat > critical_val
# Part (d): Cpk point estimate and 99% CI
Cpk_hat <- min((USL2 - xbar2)/(3*s2), (xbar2 - LSL2)/(3*s2))
z_alpha <- qnorm(0.995)
Cpk_se <- sqrt(1/(9*n2*Cpk_hat^2) + 1/(2*(n2-1)))
CI_Cpk_low <- Cpk_hat * (1 - z_alpha * Cpk_se)
```

```
CI_Cpk_high <- Cpk_hat * (1 + z_alpha * Cpk_se)
# Part (e): Approximate CI for Cpk
approx_se <- sqrt(1/(2*(n2-1)))
CI_approx_low <- Cpk_hat * (1 - z_alpha * approx_se)
CI_approx_high <- Cpk_hat * (1 + z_alpha * approx_se)
cat("Problem 2 Results:\n")
cat(sprintf("(a) Cp = %.3f, 95%% CI = [%.3f, %.3f]\n", Cp_hat, CI_Cp_low, CI_Cp_high))
cat(sprintf("(c) Test Cp > 1.33: %s (test stat=%.2f, critical=%.2f)\n",
        ifelse(reject_H0, "Reject H0", "Fail to reject H0"), test_stat, critical_val))
cat(sprintf("(d) Cpk = %.3f, 99%% CI = [%.3f, %.3f]\n", Cpk_hat, CI_Cpk_low, CI_Cpk_high))
cat(sprintf("(e) Approx CI = [%.3f, %.3f], Width Ratio = %.2f\n\n",
        CI_approx_low, CI_approx_high,
        (CI_approx_high - CI_approx_low)/(CI_Cpk_high - CI_Cpk_low)))
```

Problem 3: Process Capability

```
xbar3 <- 75
sbar3 <- 2
n3 <- 5
USL3 <- 90
LSL3 <- 70
target3 <- 80
# Estimate sigma
c4 <- 0.9400
sigma_hat3 <- sbar3 / c4
# Part (a): Potential capability
Cp3 <- (USL3 - LSL3) / (6 * sigma_hat3)
# Part (b): Actual capability
Cpk3 <- min((USL3 - xbar3)/(3*sigma_hat3), (xbar3 - LSL3)/(3*sigma_hat3))
# Part (c): Reduction in fallout
fallout_original <- pnorm(LSL3, xbar3, sigma_hat3) +
            (1 - pnorm(USL3, xbar3, sigma_hat3))
fallout_nominal <- pnorm(LSL3, target3, sigma_hat3) +
            (1 - pnorm(USL3, target3, sigma_hat3))
reduction <- fallout_original - fallout_nominal
cat("Problem 3 Results:\n")
cat(sprintf("(a) Cp = %.3f\n", Cp3))
cat(sprintf("(b) Cpk = %.3f\n", Cpk3))
cat(sprintf("(c) Fallout reduction: %.4f (%.1f%% reduction)\n\n",
        reduction, 100*reduction/fallout_original))
```

Problem 4: Process Comparison

```
# Process A
xbarA <- 100
sbarA <- 3
sigmaA <- sbarA / c4
CpA <- (112 - 88) / (6 * sigmaA)
CpkA <- min((112 - xbarA)/(3*sigmaA), (xbarA - 88)/(3*sigmaA))
falloutA <- pnorm(88, xbarA, sigmaA) + (1 - pnorm(112, xbarA, sigmaA))
# Process B
xbarB <- 105
```

```
sbarB <- 1
sigmaB <- sbarB / c4
CpB <- (112 - 88) / (6 * sigmaB)
CpkB <- min((112 - xbarB)/(3*sigmaB), (xbarB - 88)/(3*sigmaB))
falloutB <- pnorm(88, xbarB, sigmaB) + (1 - pnorm(112, xbarB, sigmaB))
cat("Problem 4 Results:\n")
cat("Process A:\n")
cat(sprintf("  Cp = %.3f, Cpk = %.3f, Fallout = %.6f\n", CpA, CpkA, falloutA))
cat("Process B:\n")
cat(sprintf("  Cp = %.3f, Cpk = %.3f, Fallout = %.6f\n", CpB, CpkB, falloutB))
cat("Conclusion: Process B is preferred due to lower fallout\n\n")
```

```
# Problem 5: Tolerance Interval Sample Size
alpha <- 0.01
gamma <- 0.95
chi_val <- qchisq(1 - alpha, 4)
n5 <- 0.5 + ((2 - alpha)/alpha) * (chi_val/4)
cat("Problem 5 Result:\n")
cat(sprintf("Required sample size: %d\n\n", ceiling(n5)))
```

```
# Problem 6: Tolerance Intervals
n6 <- 25
xbar6 <- 85
s6 <- 10
k <- 1.838  # Tolerance factor for 95% confidence, 90% coverage
# (i) Upper tolerance limit
UTL <- xbar6 + k * s6
# (ii) Lower tolerance limit
LTL <- xbar6 - k * s6
cat("Problem 6 Results:\n")
cat(sprintf("(i) 95%% of measurements below: %.2f\n", UTL))
cat(sprintf("(ii) 95%% of measurements above: %.2f\n\n", LTL))
```

```
# Problem 7: CUSUM Chart
data7 <- c(8, 8.01, 8.02, 8.01, 8.06, 8.07, 8.01, 8.04, 8.02, 8.01,
           8.05, 8.04, 8.03, 8.05, 8.06, 8.04, 8.05, 8.06, 8.04, 8.02, 8.03, 8.05)
target7 <- 8.02
sigma7 <- 0.05
k7 <- 0.5
h7 <- 4.77
H <- h7 * sigma7
K <- k7 * sigma7
# Initialize CUSUM
Cplus <- numeric(length(data7))
Cminus <- numeric(length(data7))
for (i in 1:length(data7)) {
  if (i == 1) {
    Cplus[i] <- max(0, data7[i] - (target7 + K))
    Cminus[i] <- max(0, (target7 - K) - data7[i])
  } else {
    Cplus[i] <- max(0, data7[i] - (target7 + K) + Cplus[i-1])
```

```
  Cminus[i] <- max(0, (target7 - K) - data7[i] + Cminus[i-1])
 }
}
# Check for out-of-control
out_of_control <- sum(Cplus > H | Cminus > H)
# Part (b): Estimate sigma from moving range
MR <- abs(diff(data7))
MRbar <- mean(MR)
sigma_hat7 <- MRbar / 1.128
cat("Problem 7 Results:\n")
cat(sprintf("(a) Number of out-of-control points: %d\n", out_of_control))
cat(sprintf("(b) Estimated sigma = %.4f (given sigma = 0.05 is %s)\n\n",
        sigma_hat7, ifelse(abs(sigma_hat7 - 0.05) > 0.01, "unreasonable", "reasonable")))
```

Problem 8: CUSUM ARL

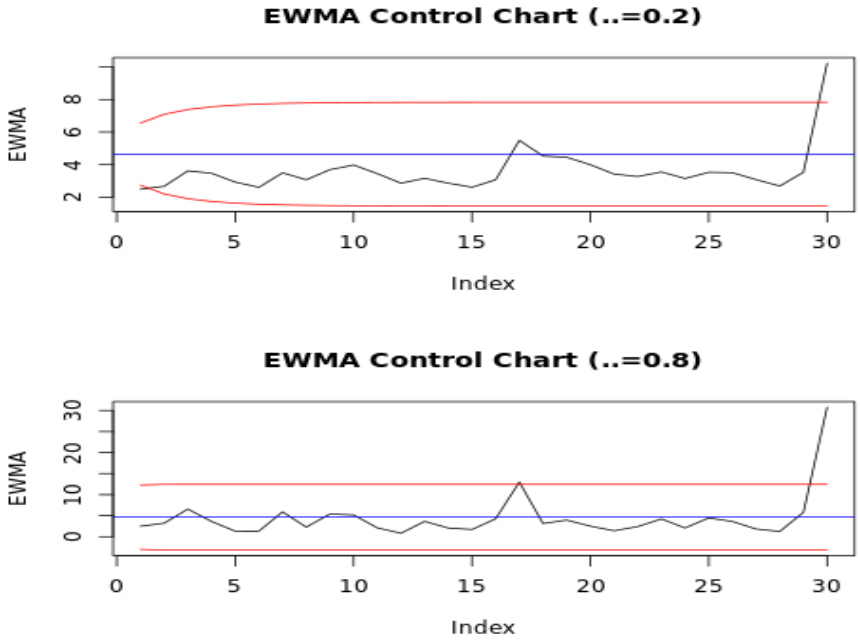
```
h8 <- 7
k8 <- 0.4
delta <- 0.7
# In-control ARL (delta=0)
b <- h8 + 1.166
ARL0 <- (exp(-2*(-k8)*b) - 1 + 2*(-k8)*b) / (2*(-k8)^2)
# Out-of-control ARL (delta=0.7)
delta_plus <- delta - k8
delta_minus <- -delta - k8
ARL1_plus <- (exp(-2*delta_plus*b) - 1 + 2*delta_plus*b) / (2*delta_plus^2)
ARL1_minus <- ifelse(delta_minus < 0,
                (exp(-2*delta_minus*b) - 1 + 2*delta_minus*b) / (2*delta_minus^2),
                Inf)
ARL1 <- 1 / (1/ARL1_plus + 1/ARL1_minus)
beta <- (ARL1 - 1) / ARL1
detect_prob <- 1 - beta
cat("Problem 8 Results:\n")
cat(sprintf("In-control ARL: %.1f\n", ARL0))
cat(sprintf("Out-of-control ARL: %.1f\n", ARL1))
cat(sprintf("Probability of detecting shift: %.3f\n\n", detect_prob))
```

Problem 9: EWMA Control Chart

```
wait_times <- c(2.49, 3.39, 7.41, 2.88, 0.76, 1.32, 7.05, 1.37, 6.17, 5.12,
            1.34, 0.50, 4.35, 1.67, 1.63, 4.88, 15.19, 0.67, 4.14, 2.16,
            1.14, 2.66, 4.67, 1.54, 5.06, 3.40, 1.38, 1.11, 6.92, 36.99)
lambda1 <- 0.2
lambda2 <- 0.8
L9 <- 2.5
# Estimate parameters
MR_wait <- abs(diff(wait_times))
MRbar_wait <- mean(MR_wait)
sigma_wait <- MRbar_wait / 1.128
mu_wait <- mean(wait_times)

# EWMA function
```

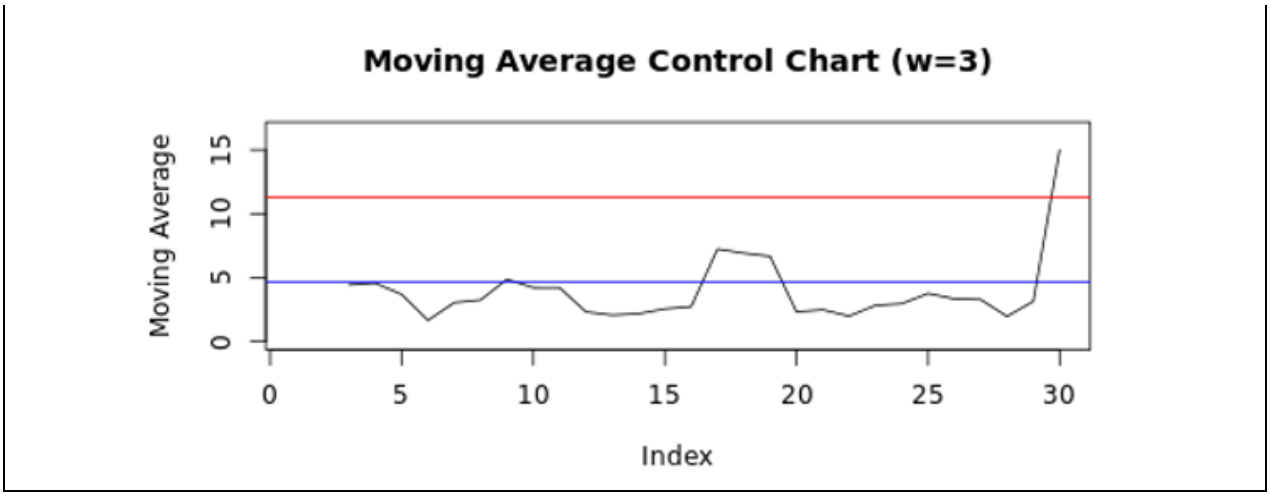
```
ewma <- function(data, lambda) {
  z <- numeric(length(data))
  z[1] <- data[1]
  for (i in 2:length(data)) {
    z[i] <- lambda * data[i] + (1 - lambda) * z[i-1]
  }
  return(z)
}
# Control limits function
ewma_limits <- function(lambda, L, sigma, n, mu) {
  lcl <- numeric(n)
  ucl <- numeric(n)
  for (i in 1:n) {
    factor <- L * sigma * sqrt((lambda/(2 - lambda)) * (1 - (1 - lambda)^(2*i)))
    lcl[i] <- mu - factor
    ucl[i] <- mu + factor
  }
  return(list(lcl = lcl, ucl = ucl))
}
# Calculate EWMA and limits
ewma1 <- ewma(wait_times, lambda1)
ewma2 <- ewma(wait_times, lambda2)
limits1 <- ewma_limits(lambda1, L9, sigma_wait, length(wait_times), mu_wait)
limits2 <- ewma_limits(lambda2, L9, sigma_wait, length(wait_times), mu_wait)
# Plot results
par(mfrow = c(2, 1))
plot(ewma1, type = "l", ylim = range(c(ewma1, limits1$lcl, limits1$ucl)),
     main = "EWMA Control Chart (λ=0.2)", ylab = "EWMA")
lines(limits1$lcl, col = "red")
lines(limits1$ucl, col = "red")
abline(h = mu_wait, col = "blue")
plot(ewma2, type = "l", ylim = range(c(ewma2, limits2$lcl, limits2$ucl)),
     main = "EWMA Control Chart (λ=0.8)", ylab = "EWMA")
lines(limits2$lcl, col = "red")
lines(limits2$ucl, col = "red")
abline(h = mu_wait, col = "blue")
cat("Problem 9: See plots for EWMA control charts\n")
cat("Effect of λ: Larger λ widens control limits, smaller λ tightens limits\n\n")
```



Problem 10: Moving Average Control Chart

```
w <- 3
ma <- function(data, w) {
  n <- length(data)
  ma_vals <- rep(NA, n)
  for (i in w:n) {
    ma_vals[i] <- mean(data[(i-w+1):i])
  }
  return(ma_vals)
}

ma_vals <- ma(wait_times, w)
sigma_ma <- sigma_wait / sqrt(w)
ucl_ma <- mu_wait + 3 * sigma_ma
lcl_ma <- mu_wait - 3 * sigma_ma
# Plot
plot(ma_vals, type = "l", ylim = c(0, max(ma_vals, na.rm = TRUE) * 1.1),
     main = "Moving Average Control Chart (w=3)", ylab = "Moving Average")
abline(h = mu_wait, col = "blue")
abline(h = ucl_ma, col = "red")
abline(h = lcl_ma, col = "red")
# Check control
in_control <- all(ma_vals >= lcl_ma & ma_vals <= ucl_ma, na.rm = TRUE)
cat("Problem 10: See plot for Moving Average chart\n")
cat(sprintf("Process is %s control\n", ifelse(in_control, "in", "out of")))
cat("Comparison with EWMA: MA chart is less sensitive to small shifts\n")
```


Moving Average Control Chart (w=3)
Moving Average
0
5
10
15
0
5
10
15
20
25
30
Index